

This document serves as the primary knowledge base for the InternMatch AI Product Assistant Chatbot. It covers the product overview, all features, usage guide, architecture, and technology stack.

1. Product Overview

Product Name

InternMatch AI

Product Description

InternMatch AI is an AI-powered internship matching platform that helps students and fresh graduates find the most relevant internship opportunities. It uses Retrieval-Augmented Generation (RAG) and large language models (LLMs) to intelligently parse resumes, match candidates to internships, generate personalised cover letters, and identify skill gaps · all in one unified dashboard.

Problem the Product Solves

Finding internships is time-consuming and imprecise. Students spend hours manually reading job descriptions and tailoring applications. Recruiters spend equal time sifting through irrelevant resumes. InternMatch AI eliminates this friction by:

- o Automatically parsing and understanding resume content using AI
- o Semantically matching candidates to relevant internships using vector similarity
- o Generating personalised cover letters in seconds
- o Showing candidates exactly which skills they need to develop for a target role

Target Users

- o Students and fresh graduates seeking internships relevant to their skills and academic background
- o Recruiters and hiring managers who want to surface the most suitable candidates quickly
- o Career counsellors helping students navigate internship applications

Main Objectives

- o Automate resume parsing and structured data extraction using LLMs
- o Perform semantic internship matching using FAISS vector similarity search
- o Generate personalised, professional cover letters using AI
- o Identify and communicate skill gaps between the candidate profile and job requirements
- o Provide a conversational Product Assistant to answer user questions about the platform

2. Product Features and Functionalities

Feature 1: Resume Upload and AI Parsing

Feature Name: Resume Upload and AI Parsing

Purpose:

Allow users to submit their resume so the system can extract structured information (name, skills, education, experience, projects) and use it as the basis for all AI-powered features.

How It Works:

- o The user uploads a PDF resume through the dashboard
- o PyMuPDF extracts the raw text from the PDF
- o The extracted text is sent to Groq LLM (llama-3.1-8b-instant) with a structured extraction prompt

- o The LLM returns a JSON object with: name, skills (list), education, experience, projects
- o The structured data is stored in PostgreSQL linked to the user account
- o Background processing immediately triggers the AI matching engine

How Users Can Use It:

- o Log in to the InternMatch AI dashboard
- o Click the "Upload Resume" button on the Overview tab
- o Select a PDF file from your device
- o Click "Upload and Analyse"
- o Watch the live progress indicator: Extracting text · Parsing with AI · Running matching engine

Expected Output or Result:

- o Confirmation that the resume was parsed successfully
- o Displayed extracted data: name, skills list, education, experience summary, projects
- o AI matching begins automatically in the background

Feature 2: AI Internship Matching

Feature Name: AI Internship Matching

Purpose:

Surface the most semantically relevant internship opportunities for the user by comparing their resume against all available listings using vector similarity search.

How It Works:

- o The user resume data (skills, education, experience, projects) is converted into a vector embedding using sentence-transformers/all-MiniLM-L6-v2
- o This embedding is compared against pre-indexed internship listing embeddings stored in a FAISS vector database
- o The top 3 most similar internship listings are retrieved using L2 distance
- o The retrieved listings are passed to Groq LLM along with the candidate summary
- o The LLM generates a natural-language matching rationale explaining why each listing fits
- o Results are cached in PostgreSQL for instant retrieval on return visits

How Users Can Use It:

- o Upload your resume (matching runs automatically after upload)
- o Navigate to the Overview tab on the dashboard
- o Your top matched internships appear with company name, job title, and relevance score
- o Click on any match to view full details

Expected Output or Result:

- o A ranked list of the top 3 most relevant internship opportunities
- o Each result shows: Company, Job Title, Relevance Score (0-100%)
- o An AI-generated rationale paragraph explaining the match

Feature 3: Cover Letter Generator

Feature Name: Cover Letter Generator

Purpose:

Automatically produce a professional, personalised cover letter tailored to a specific internship listing.

How It Works:

- o The user selects an internship (company and job title)
- o The system retrieves the full internship description from the dataset

- o The user resume data is combined with the internship details into a prompt
- o Google Gemini (gemini-2.5-flash) generates the cover letter · chosen for long-form creative writing quality
- o If Gemini is rate-limited, the system falls back to Groq automatically

How Users Can Use It:

- o Navigate to the Opportunities tab
- o Find any internship listing
- o Click "Generate Cover Letter"
- o Wait 3-8 seconds for generation
- o Read, copy, or use the generated letter

Expected Output or Result:

- o A complete, ready-to-send cover letter (3-4 paragraphs)
- o Uses the candidate real name (no placeholders)
- o Personalised to the specific company and role
- o Highlights the candidate relevant skills from their resume

Feature 4: Skill Gap Analysis

Feature Name: Skill Gap Analysis

Purpose:

Show candidates exactly which skills they are missing for a target internship role, enabling targeted learning and development.

How It Works:

- o The user selects a specific internship
- o The system retrieves the full internship requirements
- o The candidate skills from their resume are compared against the job requirements
- o Groq LLM identifies specific gaps and returns a concise bullet-point list

How Users Can Use It:

- o Navigate to the Opportunities tab
- o Click "Generate Insights" on any internship listing
- o View the Skill Gap section in the results panel

Expected Output or Result:

- o A bullet-point list of specific skills the candidate lacks for that role
- o Actionable and concise items (e.g. "Experience with Docker containerisation")
- o Generated within 2-5 seconds

Feature 5: Product Assistant Chatbot

Feature Name: Product Assistant Chatbot

Purpose:

Provide an intelligent conversational assistant that answers any question about InternMatch AI · how to use it, how it works, its architecture, and technology · using this product knowledge document as its knowledge source.

How It Works:

- o The user types a question in the chat interface
- o The question is converted to a vector embedding using sentence-transformers/all-MiniLM-L6-v2
- o The embedding searches a FAISS index built from this Product Knowledge Document
- o The top 3 most relevant document chunks are retrieved

- o The previous conversation history (last 10 messages) is retrieved from PostgreSQL for this user and session
- o A prompt is built: System Instructions + Conversation History + Retrieved Context + Current Question
- o Groq LLM (llama-3.1-8b-instant) generates the response
- o The response is returned and the full conversation is stored in the database

How Users Can Use It:

- o Log in and click the "Product Assistant" tab in the dashboard sidebar
- o Type any question about InternMatch AI
- o Press Enter or click Send
- o The chatbot responds using the product documentation
- o Ask follow-up questions · the chatbot remembers the conversation within the session
- o Start a new session anytime with "New Chat"

Expected Output or Result:

- o Accurate, document-grounded answers to product questions
- o Responses maintain context across multiple turns in a session
- o Each session is isolated per user · no cross-user data leakage
- o Optional source citations showing which document section was used

3. How to Use the Product

General Flow

- o User logs into the application
- o User accesses the product dashboard
- o User selects the required functionality from the sidebar
- o User uploads or enters the required information
- o The system processes the request using AI
- o The user receives the final result

Detailed Step-by-Step Guide

Step 1 · Register an Account

- o Visit the InternMatch AI homepage
- o Click "Get Started" or "Register"
- o Enter your email address and a password
- o Submit the form
- o You are automatically redirected to the dashboard

Step 2 · Upload Your Resume

- o On the dashboard Overview tab, find the "Upload Resume" section
- o Click "Choose File" and select your PDF resume
- o Click "Upload and Analyse Resume"
- o A live progress bar shows: Extracting text · Parsing with AI · Running AI matching
- o Once complete, your parsed resume details appear on the screen

Step 3 · View Your AI-Matched Internships

- o Your top 3 matched internships appear on the Overview tab after processing
- o Each match shows: Company, Title, Relevance Score
- o An AI rationale explains why each listing matches your profile

Step 4 · Browse All Opportunities

- o Click the "Opportunities" tab in the sidebar

- o Browse or search all available internship listings
- o Use the location filter to narrow results

Step 5 · Generate a Cover Letter

- o On any internship listing, click "Generate Cover Letter"
- o Wait a few seconds for the AI to generate your personalised letter
- o Read and copy the generated text

Step 6 · View Skill Gap Analysis

- o On any listing, click "Generate Insights"
- o The Skill Gap section shows exactly which skills you are missing for that role

Step 7 · Use the Product Assistant

- o Click "Product Assistant" in the sidebar
- o Type any question about InternMatch AI
- o Examples: "How does matching work?", "What is RAG?", "How do I upload a resume?"
- o Press Enter to submit
- o Ask follow-up questions · the assistant remembers the conversation

4. Product Architecture

High-Level Architecture

```
User
  |
  v
Frontend (Next.js / React)
  |
  v
Backend API (FastAPI / Python)
  |
  +-- LLM Layer (Groq + Gemini via LangChain)
  +-- Vector Database (FAISS)
  +-- Relational Database (PostgreSQL)
  +-- Document Store (Internship JSON + Product Knowledge PDF)
```

Component Responsibilities

Frontend · Next.js (React)

- o Role: User interface for the entire application
- o Renders the dashboard with tabbed navigation (Overview, Opportunities, Profile, Settings, Product Assistant)
- o Handles file upload, button clicks, and API calls
- o Displays AI-generated results to the user
- o Communicates with the backend via HTTP REST API

Backend · FastAPI (Python)

- o Role: Business logic layer and API gateway
- o Handles user registration, login, resume upload, and result retrieval
- o Orchestrates AI operations: triggers LLM parsing, FAISS search, and LLM generation
- o Manages database reads and writes
- o Runs long operations as background tasks to keep the API responsive

LLM Layer · Groq + Google Gemini via LangChain

- o Role: AI intelligence · generates all text outputs

- o Groq (llama-3.1-8b-instant): Resume parsing, matching rationale, skill gap, chatbot responses
- o Google Gemini (gemini-2.5-flash): Cover letter generation (superior creative writing quality)
- o LangChain: Orchestration framework for prompts, RAG chains, and LLM calls

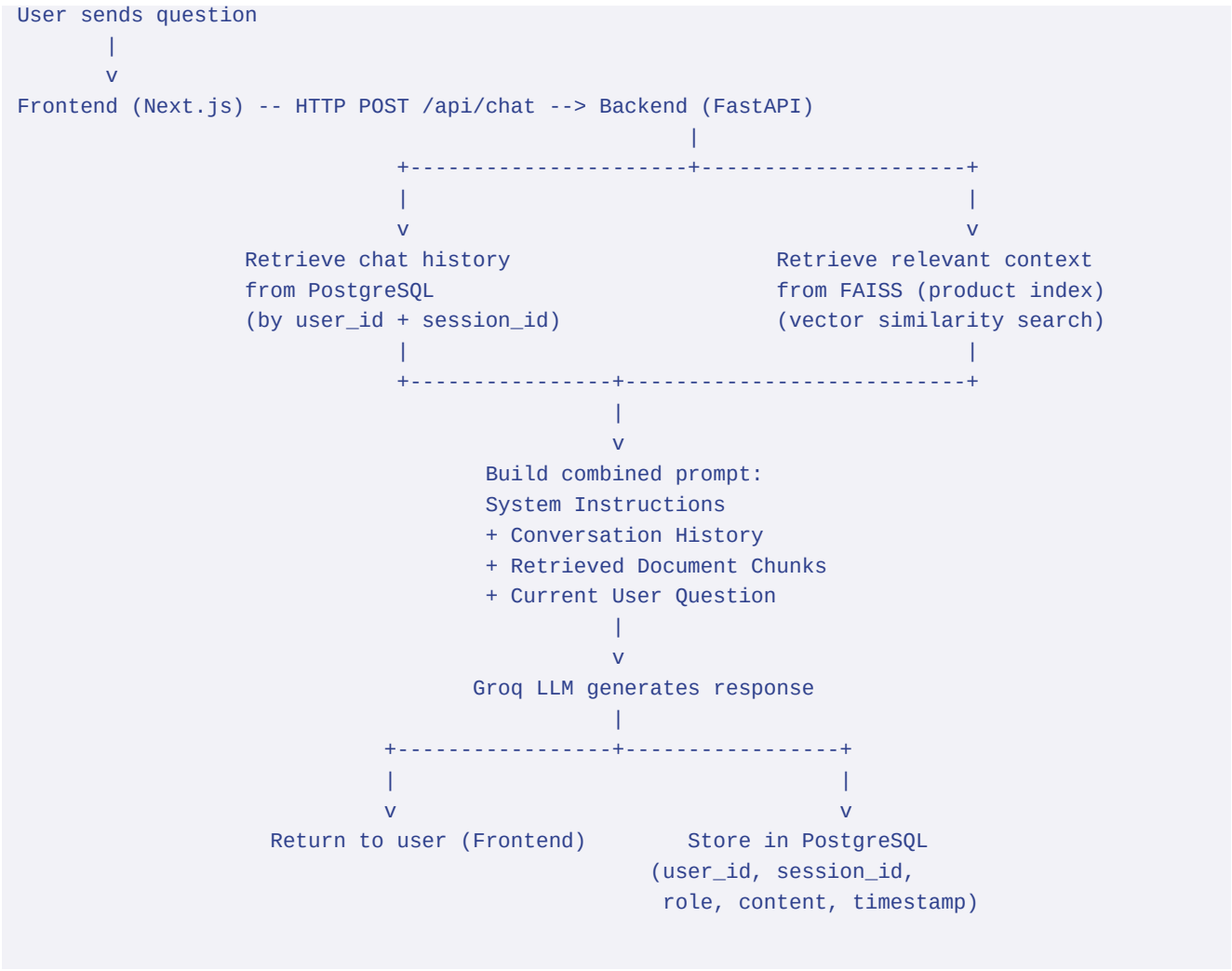
Vector Database · FAISS

- o Role: Semantic similarity search engine
- o Internship Index: Embeddings for all internship listings · used for job matching
- o Product Index: Embeddings for this knowledge document · used by the chatbot
- o Embedding model: sentence-transformers/all-MiniLM-L6-v2

Relational Database · PostgreSQL

- o Role: Persistent data storage
- o Stores: user accounts, uploaded resumes, parsed data, match results, chat conversations

Complete System Flow for Chatbot



5. Technology Stack

Why Each Technology Was Selected

Frontend

Technology	Reason
Next.js (React)	Server-side rendering for fast initial page l
TypeScript	Catches type errors at compile time. Improves

Backend

Technology	Reason
Python	Dominant language for AI/ML. All major LLM an
FastAPI	Async-first, high-performance Python web fram
Pydantic	Data validation for all API request and respo
Uvicorn	ASGI server for running FastAPI in production

AI and LLM

Technology	Reason
LangChain	Provides ready-made abstractions for RAG chai
Groq (llama-3.1-8b-instant)	Chosen over OpenAI: free tier with generous l
Google Gemini (gemini-2.5-flash)	Superior quality for long-form creative writi
sentence-transformers/all-MiniLM-L6-v2	Free, local embedding model. Produces high-qu

Vector Database

Technology	Reason
FAISS (Facebook AI Similarity Search)	High-performance vector similarity search. Ru

Relational Database

Technology	Reason
PostgreSQL	Reliable, production-grade relational databas
SQLAlchemy	Python ORM providing clean model definitions

Document Processing

Technology	Reason
PyMuPDF	Fast, reliable PDF text extraction. Used for

6. API Reference

Authentication

Method	Endpoint	Description
POST	/api/register	Register a new user account
POST	/api/login	Login and receive user_id

Resume Management

Method	Endpoint	Description
POST	/api/upload_resume?user_id={id}	Upload and AI-parse a PDF resume
GET	/api/resumes/{user_id}	Get all resumes for a user
POST	/api/resumes/{resume_id}/activate?user_id={id}	Set a resume as active
GET	/api/analysis_status/{user_id}	Poll live progress of background analysis

Matching and Insights

Method	Endpoint	Description
GET	/api/matches/{user_id}?requesting_user_id={id}	Get AI match results
POST	/api/retry_matching/{user_id}?requesting_user_id={id}	Retry failed matching
POST	/api/generate_insights	Generate cover letter and skill gap
GET	/api/opportunities	Get all internship listings

Product Chatbot (new feature)

Method	Endpoint	Description
POST	/api/chat/session	Create a new chat session, returns session_id
GET	/api/chat/sessions/{user_id}	List all chat sessions for a user
POST	/api/chat	Send a message, receive AI response
GET	/api/chat/history/{user_id}/{session_id}	Get full conversation history

7. Database Schema

users table

Column	Type	Description
id	Integer (PK)	Unique user identifier
email	String (unique)	User email address
hashed_password	String	Bcrypt-hashed password
full_name	String	User full name
phone	String	Contact number
university	String	Institution name

resumes table

Column	Type	Description
id	Integer (PK)	Unique resume identifier
user_id	Integer (FK)	References users.id
filename	String	Original file name
raw_text	Text	Full extracted PDF text
structured_data	Text (JSON)	Parsed fields: name, skills, education, exper
match_result	Text (JSON)	Cached FAISS and LLM matching results
is_active	Boolean	Whether this is the current active resume
created_at	DateTime	Upload timestamp

chat_messages table

Column	Type	Description
id	Integer (PK)	Unique message identifier
user_id	Integer (FK)	References users.id
session_id	String (indexed)	UUID identifying the chat session
role	String	"user" or "assistant"

content	Text	Message content
created_at	DateTime	Message timestamp
source_chunks	JSON	Document chunks used for this response
feedback	String	User feedback: "like", "dislike", or null

8. Setup and Configuration

Prerequisites

- o Python 3.10 or higher
- o Node.js 18 or higher
- o PostgreSQL 14 or higher

Environment Variables

Create a .env file in the project root:

```
GROQ_API_KEY=your_groq_api_key_here
GEMINI_API_KEY=your_gemini_api_key_here
DATABASE_URL=postgresql://postgres:password@localhost:5432/InternshipApp
```

Backend Setup

1. Create virtual environment: `python -m venv venv`
2. Activate: `venv\Scripts\activate`
3. Install dependencies: `pip install -r backend/requirements.txt`
4. Run migrations: `python backend/migrate.py`
5. Build chatbot FAISS index: `python backend/chatbot/doc_indexer.py`
6. Start server: `uvicorn backend.main:app --reload --port 8000`

Frontend Setup

1. Install dependencies: `cd frontend && npm install`
2. Start dev server: `npm run dev`

Access Points

- o Application: <http://localhost:3000>
- o API: <http://localhost:8000>
- o API Documentation: <http://localhost:8000/docs>